

# Final Project — Preliminary Proposal

Course: 3917 — AI Techniques for Malware Detection Date: 2026-06-15

---

## 1. Group Members

---

- Ben Volovelsky — ben.volovelsky@post.runi.ac.il
  - Noam Delbari — noam.delbari@post.runi.ac.il
- 

## 2. Chosen Attack Technique

---

**Attack:** Detection of **malicious Linux shell commands** — a *Living-off-the-Land (LotL)* technique in which adversaries abuse the trusted, pre-installed Unix shell and legitimate system binaries (`bash`, `curl`, `nc`, `python`, `awk`, ...) to execute payloads "fileless-ly" — spawning reverse shells, downloading-and-executing, enumerating, and escalating — instead of dropping conventional malware that signature engines would catch.

**Primary MITRE ATT&CK ID:**

- **T1059.004** — Command and Scripting Interpreter: **Unix Shell** (*primary*)

**Associated behaviours covered:**

- **Reverse / bind shells** — outbound interactive shells (relates to C2, T1071).
- **LotL binary abuse** — shell escapes and download-exec via legitimate binaries (the Linux analogue of LOLBins; catalogued by *GTFOBins*).
- **Indicator removal** — history wiping / log tampering (T1070) as defense evasion.

**Why this technique.** Living-off-the-Land is now a dominant intrusion style: adversaries increasingly avoid custom malware and instead drive built-in interpreters, which are signed, ubiquitous, and invisible to file-based scanning. On Linux servers — the backbone of cloud and container infrastructure — this plays out almost entirely through the shell, yet it is far less studied than its Windows/PowerShell counterpart. It is detectable directly from standard host telemetry (auditd command-line auditing), maps cleanly to MITRE ATT&CK, and is backed by *public, labelled* data (assembled from public sources, see §4) and recent top-venue literature (see §5) — without the "obfuscated = malicious" confound that plagues PowerShell corpora.

---

## 3. Attack Characteristics & Log Mapping

---

### 3.1 Behavioural characteristics of malicious Linux shell activity

| Behaviour                       | How it is used in the attack                                                                                                                               | How it is reflected in host logs                                                                                                                                                                                                                                                                                                                     |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reverse / bind shell            | Gives the attacker an interactive remote session right after initial access (reverse = beacon out to C2; bind = listen for inbound). The primary foothold. | auditd EXECVE records the spawning command ( <code>bash -i</code> , <code>nc -e /bin/sh</code> , <code>python ... pty.spawn</code> ) and the <code>/dev/tcp/&lt;ip&gt;/&lt;port&gt;</code> redirect; an unusual parent (e.g. a web service spawning <code>bash</code> ) stands out. Sysmon-for-Linux Event 1 logs the same process and command line. |
| Download-and-execute (fileless) | Pulls the next-stage payload from attacker infrastructure and runs it from memory, leaving no file on disk for AV to scan. Delivery / staging.             | auditd EXECVE shows a fetch tool ( <code>curl</code> , <code>wget</code> , <code>python urllib</code> ) whose output is piped straight into <code>bash / sh</code> ; the attacker URL appears in <code>argv</code> .                                                                                                                                 |

| Behaviour                          | How it is used in the attack                                                                                                                                                                                                                    | How it is reflected in host logs                                                                                                                                                                                                                                            |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| LotL binary abuse (GTFOBins)       | Uses a trusted, allow-listed binary ( <code>awk</code> , <code>find</code> , <code>vim</code> , <code>perl</code> ...) to spawn a shell, read protected files, or escalate, hiding the action behind a legitimate program. Execution + evasion. | <code>auditd EXECVE</code> captures the binary running with its tell-tale escape argument ( <code>awk 'BEGIN{system("/bin/sh")}'</code> , <code>find ... -exec /bin/sh</code> ), i.e. a normal tool invoked in an abnormal way.                                             |
| Encoded / inline execution         | Hides the real command from quick inspection and simple string rules by base64-encoding or assembling it inline, decoding and running it only at runtime. Light obfuscation.                                                                    | <code>auditd EXECVE</code> records the <code>base64 -d</code> decode piped into a shell, <code>eval</code> , or nested <code>\$( ... )</code> ; the encoded blob sits in <code>argv</code> and can be decoded during analysis.                                              |
| Host & account enumeration         | Discovery: maps users, host, network and privileges to plan privilege escalation and lateral movement.                                                                                                                                          | <code>auditd EXECVE</code> shows a burst of <code>whoami</code> , <code>id</code> , <code>uname -a</code> , <code>cat /etc/passwd</code> , <code>ss</code> , <code>netstat</code> from one session — each benign alone, suspicious as a cluster within seconds.             |
| Privilege escalation / persistence | Raises privileges ( <code>sudo</code> , <code>setuid</code> ) and plants a foothold that survives reboot ( <code>cron</code> , <code>systemd</code> unit, shell rc files) to keep access.                                                       | <code>auditd EXECVE</code> plus file-write records show <code>chmod +s</code> , <code>crontab -e</code> , writes under <code>/etc/cron.*</code> , <code>systemctl enable</code> , edits to <code>~/.bashrc</code> ; <code>auth.log</code> logs the <code>sudo / su</code> . |
| Defense evasion                    | Covers tracks by clearing history and tampering with logs so the intrusion is harder to spot and investigate. Anti-forensics.                                                                                                                   | The evasion commands are themselves recorded by <code>auditd</code> independently of shell history — <code>history -c</code> , <code>unset HISTFILE</code> , <code>chattr</code> , truncating <code>/var/log/...</code> appear in <code>EXECVE</code> .                     |

### 3.2 Host-log sources

Our detection input is the **raw command-line text** (`argv`). The reflections in the table above are recorded across several standard, default-available Linux telemetry sources:

| Telemetry source          | Record / Event                                                | What it provides                                                                                            |
|---------------------------|---------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Linux <code>auditd</code> | <code>type=EXECVE</code> (+ <code>SYSCALL</code> )            | Full <code>argv</code> of every executed command — the richest signal (Linux analogue of Windows 4688/4104) |
| Sysmon for Linux          | Event ID 1 (Process Creation)                                 | Command line, parent process, image                                                                         |
| Shell history             | <code>~/.bash_history</code> ,<br><code>~/.zsh_history</code> | Interactive command record                                                                                  |
| Authentication log        | <code>/var/log/auth.log</code> ,<br><code>secure</code>       | <code>sudo / su</code> and SSH session context                                                              |
| eBPF / EDR sensors        | <code>execve</code> hooks                                     | Real-time process-execution telemetry                                                                       |

This is unambiguously **host-log telemetry**: the command text required for detection is recorded by default-available Linux logging (`auditd` command-line auditing, Sysmon for Linux, shell history).

## 4. Target Datasets (two, publicly available)

No single public dataset carries both malicious and benign Linux shell commands at scale — the benign side, at scale, is consistently private. We therefore **assemble two both-class datasets from public sources** (standard practice in this area: SLP 2021, Oliveira & Café 2024). Each dataset contains attack commands and

normal commands, and **each is split 80/20 into a training and a test set**. They differ deliberately in the *realism* of their data: Dataset 1 is **generated and curated**, Dataset 2 is **real** activity captured in the wild. This lets us measure detection separately on each regime, and transfer between the two.

The two datasets are **provenance-matched**: within a dataset, the attack side and the normal side are drawn from the same *kind* of source. This matters. If we paired real honeypot attacks with hand-written tutorial benigns, a model could separate the classes on collection style alone and never learn anything about maliciousness. Matching provenance removes that escape route.

Class ratio is **1 attack : 3 normal** in both datasets, since malicious commands are rare in real host telemetry and a 50/50 split would overstate precision.

### Dataset 1 — Generated + curated, both sides

- **Attacks (3,812)**. **HackTricks** Linux `privesc/enum/LotL` tradecraft — the `linux-hardening/` subtree of the book (CC-BY-NC-4.0; 1,805) · **GTFOBins** LotL payloads (GPL-3.0; 957) · **Atomic Red Team** MITRE-technique test commands (MIT; 685) · **QuasarNix** synthetic LotL reverse shells (`dtrizna/QuasarNix`, Apache-2.0; 204) · **SLP** curated one-liners (`dtrizna/slp`, MIT; 101) · **Payloads** bash one-liners (`swisskyrepo/InternalAllTheThings`, MIT; 60). HackTricks and Payloads are new since the previous version of this proposal; QuasarNix — formerly 17,580 rows and ~90% of the attack side — demotes to a 5.4% slice under the uniform shape cap (239,638 raw rows → 102 distinct shapes → 204 kept).
- **Normal (11,436)**. **tldr-pages** (CC-BY-4.0; 4,576) · **bash-instruct-55k** (MIT; 3,935) · **nl2bash** (GPL-3.0; 1,567) · **LinLM linux-commands** (Apache-2.0; 752) · **bash\_command\_6k** (Apache-2.0; 606).
- **Size**. **15,248** commands — 12,199 train / 3,049 test, across 14,945 distinct command shapes.
- **Role**. In-distribution baseline on generated-and-curated data, and the basis for the **unseen-variant robustness** check (§6, E3).

### Dataset 2 — Real-world, both sides

- **Attacks (2,394)**. Real attacker commands from an **SSH honeypot** — `ML4Net/SSH-Shell-Attacks`, 233,035 Cowrie sessions (MIT). Sessions are split into atomic commands, and an atom is malicious **by provenance** — it was typed inside a real attacker session — never because it matches a keyword. Atoms are dropped only by criteria that never inspect the command text for malicious content (`not-a-command` / malformed-line filters, plus a label-noise scrub removing atoms seen only in sessions the upstream annotators fingerprinted as harmless), and botnet near-duplicates collapse under the uniform shape cap (at most two rows per shape). An earlier build instead *selected* atoms by a malicious-marker keyword list (`download`, `pipe-to-shell`, `chmod +x`, `/dev/tcp`, ...) — a circular rule a classifier could ace by re-deriving it; that list survives only as a probe/audit signal (§6.1) and decides no label.
- **Normal (7,182)**. Captured `.bash_history` files (`spignelon/bash_history`, MIT; 5,027) · **commandlinefu** real user-submitted one-liners (2,155).
- **Size**. **9,576** commands — 7,661 train / 1,915 test, across 9,391 distinct command shapes.
- **Role**. In-distribution baseline on real data, and the target of the **cross-dataset transfer** check (§6, E2).

**Independence**. No command appears in more than one dataset or in more than one split — asserted at build time in `build_dataset.py` and re-verified on the built files (2026-08-05): **0 cross-dataset command overlap; within each dataset, 0 train/test overlap by exact command and by shape**. The two datasets share no source at all — the earlier version of this proposal split one benign source (`nl2bash`) between them, which the extra benign sources now make unnecessary.

**Why Dataset 2 stays smaller**. Its attack ceiling is real, not a sampling choice: the 233k honeypot sessions yield 407,442 distinct well-formed atoms, but those collapse to only **2,455 structurally distinct** command

shapes. The remainder are randomised variants of the same botnet templates. We tested session-level labelling (treating every command in a compromised session as hostile), which reaches ~27k rows, and **rejected** it: the additions are overwhelmingly `/bin/busybox <RANDOM>` variants and leaked password strings, i.e. volume without structural diversity, plus label noise.

**Reproducibility.** `scripts/build_dataset.py` rebuilds both datasets end to end from the public sources; `docs/DATA_CARD.md` records provenance, licences and processing; `scripts/evaluate_baseline.py` reproduces the baseline and confound audit in §6.

(Content-matched references whose datasets are not openly downloadable — used as literature, not data: *SCORE* (Erdemir et al., 2024) and *ShellCore* (Alasmary et al., 2022; real IoT-malware shell commands, request-only).)

---

## 5. Academic Literature Validation (two peer-reviewed papers)

---

Our citations pair a **current state-of-the-art** method with a **foundational** shell-command representation method.

1. **Trizna, D., Demetrio, L., Biggio, B., & Roli, F. (2026).** *Robust Large-Scale Detection of Living-Off-the-Land Reverse Shells via Data Synthesis*. *ACM Transactions on Privacy and Security (TOPS)*. DOI: 10.1145/3807450 · arXiv:2402.18329.

- *Relevance:* Current SOTA and the source of Dataset 1's QuasarNix slice (see §4). Detects Linux LotL reverse shells with ML (XGBoost / MLP on one-hot + token features) and tackles data scarcity through informed synthesis, with adversarial-robustness evaluation. Directly defines our task and our unseen-variant robustness angle.

2. **Alasmary, H., Anwar, A., Abusnaina, A., Alabduljabbar, A., Abuhamad, M., Wang, A., Nyang, D., Awad, A., & Mohaisen, D. (2022).** *ShellCore: Automating Malicious IoT Software Detection by Using Shell Commands Representation*. *IEEE Internet of Things Journal*. (IEEE Xplore doc. 9446490 · arXiv:2103.14221).

- *Relevance:* Establishes the term- and character-level representation of shell commands feeding classical ML and deep learning (~99% accuracy on malicious vs benign shell commands) — the baseline pipeline our classical and embedding tiers reproduce.

(Additional current support, if a third is wanted: Erdemir et al., *SCORE*, 2024 — syntactic/AST representations for script malware; Trizna, Shell Language Processing / Active-Learning LotL command detection, *RAID* 2021.)

---

## 6. Proposed Methodology

---

**Task.** Binary classification of a **Linux shell command / command-line** as **malicious vs benign**, taking the raw `argv` text (auditd `EXECVE` / bash history) as input.

**Central question.** Dataset 1 is *generated and curated* (HackTricks, GTF0Bins, Atomic Red Team, SLP, Payloads, a residual QuasarNix slice); Dataset 2 is *real* honeypot and shell-history capture. The scientific risk is no longer that a model memorises one generator's templates — QuasarNix is down to 5.4% of Dataset 1's attack side, and the curated bulk (HackTricks + GTF0Bins) is catalogued tradecraft, not a generator — but **catalogue coverage**: published tradecraft teaches the *syntax* of techniques, and a model trained on it can still miss what real attackers actually type. Our baseline audit (§6.1) shows this gap is real but narrowing: transfer from Dataset 1 to real data reaches F1 0.584 — much improved over earlier builds, still far below the in-distribution 0.898. Our methodology therefore aims first for a strong in-distribution baseline on each dataset, then measures **generalisation to unseen variants and across the curated ↔ real gap**.

**Models (ML → DL).** Three tiers of increasing capacity, to judge how much the harder methods actually buy us:

1. **Classical baseline** — TF-IDF over character + token n-grams into Logistic Regression / XGBoost. Cheap, interpretable, and strong on short command text.
2. **Embeddings** — word2vec / fastText trained on shell tokens (argv, flags, paths) feeding a shallow classifier, to capture token *semantics* that surface n-grams miss (e.g. equivalent binaries or flag orderings).
3. **Transformer** — a fine-tuned code/text transformer (CodeBERT or DistilBERT) reading the command text directly, for contextual representation.

### Experiments.

- **E1 — Baseline.** Train and test each tier in-distribution on **each** dataset separately (Dataset 1 and Dataset 2), establishing headline scores and seeing whether the transformer's extra capacity is justified. The two datasets' results also read as a **synthetic-and-curated vs real** comparison.
- **E2 — Cross-dataset transfer.** Train on Dataset 1 and test on Dataset 2 (and the reverse). Since Dataset 1's attacks are generated/curated and Dataset 2's are real honeypot captures, this measures whether a detector trained under one realism regime holds up on the other; a large drop signals over-fitting to one source rather than learning malice in general.
- **E3 — Unseen-variant robustness.** Within Dataset 1, hold out entire reverse-shell templates/families at training time and test on them — does the detector generalise to LotL variants it never saw? (This is the angle that replaces obfuscation-robustness.)
- **E4 — Operating point.** Report TPR at very low false-positive rates (e.g.  $10^{-3}$ – $10^{-4}$ ) — the number that matters for an analyst triaging high-volume host logs.

**Setup.** Volatile tokens are normalised, but **not to constants**: each IP address and URL host is replaced by a value derived from a hash of the original. Mapping every address to a single literal (as ShellCore/QuasarNix do) is actively harmful here — an earlier build of ours mapped all IPv4 to `1.1.1.1`, and because every QuasarNix reverse shell carries an address, that one token then appeared in 100% of malicious and ~1% of benign commands and was by itself a near-perfect classifier. Hashing preserves address *variety* while still hiding the real value. We deduplicate globally by exact string, drop strings that appear on both labels, and split 80/20 **grouped by command shape** (structure with quoted strings, numbers, hashes and randomised identifiers blanked) so an identical structure can never straddle train and test. Every source is sampled **shape-stratified** — round-robin over distinct shapes, at most two rows per shape — so no source can contribute thousands of near-copies of one template (this cap is what collapses QuasarNix from 239,638 raw rows to 204).

## 6.1 Preliminary baseline and confound audit

We ran a deliberately simple baseline (TF-IDF char 3–5-grams + logistic regression) not to report a headline score but to check the datasets are not trivially separable by an artefact. Reproduce with `scripts/evaluate_baseline.py`.

| Setting                                               | Dataset 1 (F1) | Dataset 2 (F1) |
|-------------------------------------------------------|----------------|----------------|
| In-distribution (full test)                           | 0.8975         | 0.8824         |
| Ablation: addresses stripped (full test)              | 0.8990         | 0.8843         |
| Ablation: length-matched 1:1 subset                   | 0.9301         | 0.9180         |
| Ablation: both                                        | 0.9336         | 0.9180         |
| Shortcut probe: "contains an IPv4" alone (test split) | 0.1745         | 0.0960         |

| Setting                                           | Dataset 1 (F1) | Dataset 2 (F1) |
|---------------------------------------------------|----------------|----------------|
| Shortcut probe: command length alone (test split) | 0.3858         | 0.3795         |

(The length-matched rows are a **1:1 matched subset** — prevalence 0.50 against 0.25 on the full test — so their F1 sits on a different floor and is not directly comparable to the full-test rows.)

Three things this tells us, all of which shape the work ahead:

1. **Dataset 2's signal survives the artefact probes.** It holds  $F1 \approx 0.88$  (0.918 on the matched subset) with addresses stripped *and* lengths matched — and its malicious commands are actually *shorter* than its benign ones (medians 21 vs 25 chars), so length could not carry the class anyway. Its signal is genuinely about command content (its source-separability probe passes, though only just — see `docs/KNOWN_ISSUES.md`).
2. **The expansion removed the artefact separability QuasarNix created.** This claim has reversed direction: an address-only rule on Dataset 1 now scores 0.175 (was 0.938) and a length-only classifier 0.386 (was 0.858), because the malicious median length fell 221 → 39 chars and the IPv4-in-every-row property vanished once QuasarNix stopped dominating the attack side. One honest caveat: the source-separability probe (P8) still **FAILs** on Dataset 1, because the residual 204 QuasarNix rows are all IPv4-bearing and SLP's one-liners carry similar giveaway tokens — quarantined, documented shortcuts (`docs/KNOWN_ISSUES.md`), not hidden ones.
3. **The per-source gap has largely closed.** Recall on Dataset 1's test split now spans 0.89–1.00 across all six attack sources (atomic\_red\_team 0.902, gtfobins 0.905, hacktricks 0.890; slp 0.957, payloads and quasarnix 1.000 — slp and payloads on small n of 23 and 15, quasarnix on n=58). No single generator carries the model any more. The flip side: the harder, de-QuasarNix'd mix pushes per-source benign false-positive rates up — bash6k 0.155, linlm 0.110, nl2bash 0.082, against tldr 0.029 and bash\_instruct 0.003 — so "benign FPR low throughout" is no longer true either. The commitment stands: we report **per-source recall and per-source FPR**, never the aggregate alone.

Cross-dataset transfer is where the expansion paid off most: Dataset 1 → 2 now gives F1 **0.584** (0.365 in the build the previous version of this proposal cited; 0.479 after the label and split fixes, immediately before the source expansion) — broader, more realistic curated attacks genuinely generalise better to real honeypot activity. The reverse stays weak: Dataset 2 → 1 gives 0.274 — no longer the ~0 of earlier builds, but a honeypot-trained model still recognises little of the broader tradecraft. Both directions sit far below in-distribution, which is precisely the gap E2 exists to characterise, and closing it is the substantive contribution we are aiming at.

**Metrics.** F1, ROC-AUC, PR-AUC, and **FPR at fixed recall / TPR at low FPR**. Benchmark against the QuasarNix detectors and the ShellCore term/char baseline.

**Stack.** Python · scikit-learn · gensim · PyTorch / HuggingFace.